

Programmmentwurf für Parallel Programming

MGH-TINF23 – 6. Semester

1 Organisatorisch

Die Abgabe des Programmmentwurfs inkl. aller geforderten Teile muss bis spätestens **31. Mai 2026 23:59 Uhr** in unserem **Moodle-Kursraum**¹ erfolgen. Beachten Sie hierzu auch die Abgabehinweise unter 4.

Tipp: Wir empfehlen für die Code-Entwicklung ein Git-Repository zu verwenden. Dadurch kann der programmierte Code während der Entwicklung gesichert und der Programmcode bei Problemen auch auf eine vorherige Version zurückgesetzt werden.

2 Programmieraufgabe

2.1 Kontext

Browser sind für uns Menschen das Mittel der Wahl zum Abruf von Webseiten. Um Webseiten maschinell auszulesen, werden sogenannte *Webcrawler* eingesetzt. Ein Webcrawler ist ein Programm, welches die Inhalte von Webseiten automatisiert ausliest, durchsucht und analysiert. Webcrawler eignen sich besonders um eine große Anzahl von Webseiten auszulesen sowie zur Ausführung automatisierter Aufgaben auf solchen Seiten.

2.2 Problembeschreibung

Es ist ein Webcrawler zu entwickeln, der Webseiten nach Bildern durchsucht und diese herunterlädt. Implementieren Sie dazu eine Klasse `ImageCrawler`, welche über den Konstruktor ein Konfigurationsobjekt erhält. Dieses Konfigurationsobjekt muss das folgende Interface `IImageCrawlerConfig` implementieren:

Listing 1: `IImageCrawlerConfig` Interface

```
1 public interface IImageCrawlerConfig {  
2  
3     int getNumberOfAllowedParallelWebsiteScans();  
4     int getNumberOfAllowedParallelImageDownloads();  
5     java.nio.file.Path getDownloadPath();  
6  
7 }
```

Die über die Konfiguration übergebenen Parameter sind im entwickelten Programm wie folgt aufzugreifen:

- `getNumberOfAllowedParallelWebsiteScans()`: Spezifiziert die Anzahl der max. parallel zu durchsuchenden Webseiten.
- `getNumberOfAllowedParallelImageDownloads()`: Spezifiziert die Anzahl der max. parallel herunterzuladenden Abbildungen.
- `getDownloadPath()`: Spezifiziert den Ordner, in welchen die Abbildungen heruntergeladen werden sollen.

Der Webcrawler `ImageCrawler` soll folgendes Interface implementieren:

Listing 2: `IImageCrawler` Interface

```
1 public interface IImageCrawler {  
2  
3     void crawl(final URI uri);  
4     boolean isIdle();  
5  
6 }
```

¹<https://moodle.mosbach.dhbw.de/course/view.php?id=21192>

Die Methoden sollen dabei folgende Implementierung bereitstellen:

Die über die Konfiguration übergebenen Parameter sind im entwickelten Programm wie folgt aufzugreifen:

- `void crawl(final URI uri)`: Ein Aufruf dieser Methode fügt eine weitere URL zu den durchsuchenden Webseiten hinzu. Solange `getNumberOfAllowedParallelWebsiteScans()` größer als die Anzahl der aktuell durchsuchten Webseiten ist, kann der Crawler direkt mit der Analyse der Webseite starten. Andernfalls schiebt der Crawler das Durchsuchen der URL auf, bis wieder ein Bearbeitungsplatz frei ist. Der Aufruf der Methode soll Thread-safe sein.
- `boolean isIdle()`: Gibt Auskunft darüber, ob der Crawler derzeit mit dem Durchsuchen von Webseiten oder dem Herunterladen von Abbildungen beschäftigt ist. Nur wenn derzeit keine Webseiten analysiert oder Abbildungen heruntergeladen werden, darf `true` zurückgegeben werden. Der Aufruf der Methode soll Thread-safe sein.

Der Webcrawler soll so implementiert werden, dass die Webseitendownloads/-analysen und Abbildungsdowndloads unabhängig voneinander parallel erfolgen können. Der Webcrawler soll alle Arten von Abbildungen erkennen und herunterladen, die als Abbildungen gem. dem `<img ...>`-Tag² im HTML-Quellcode gefunden werden. Das Abspeichern der Abbildungen soll unter dem in der Konfiguration angegebenen Pfad auf der Festplatte stattfinden. Die Speicherung soll pro übergebener Webseiten-URL in einem Unterordner erfolgen, wobei die Unterordner nach aufsteigenden Zahlen (beginnend bei 1) pro übergebener Webseite durchnummeriert werden sollen. Der Name der Original-Abbildung soll beim Speichern erhalten bleiben. Sollten Namenskonflikte auftreten, benennen Sie die zweite zu speichernde Datei mit einem Suffix wie `_2` vor der Dateiendung um.

2.3 Programmaufbau

Die Klasse `ImageCrawler` soll die Koordination des Programmablaufs übernehmen. Es sind außerdem mindestens folgende zwei weitere Klassen zu implementieren:

WebsiteAnalyzer: Diese Klasse fokussiert sich auf die Analyse der Webseiten und ist zuständig für den Download der HTML-Seiten (die nicht zu persistieren sind) sowie für das Erkennen und Extrahieren der Abbildungs-URLs. Ein Download der Abbildungen ist nicht über diese Klasse durchzuführen.

ImageDownloader: Diese Klasse ist für das Herunterladen und Abspeichern der Abbildungen auf der Festplatte zuständig.

2.4 Programmierhinweise

Achten Sie bei der Programmierung auf eine saubere Struktur. Fügen Sie Ihrem Programm – sofern sinnvoll – gerne weitere Klassen hinzu. Benennen Sie Variablen und Methoden sinnvoll und erstellen Sie für alle Methoden die entsprechenden JavaDocs.

Die zu implementierende Parallelität erfordert vermutlich Schutzmechanismen um gewisse Teile des Programmcodes oder Variablen vor parallelem Zugriff zu schützen. Achten Sie darauf, dass dieser Schutz nur soweit stattfindet, wie es unbedingt notwendig ist.

2.5 Rahmenbedingungen

Das Programm soll in Java entwickelt sein. Als Build-System soll Maven zum Einsatz kommen. Fremdbibliotheken (Dependencies) dürfen insbesondere als HTTP-Client und zum Einlesen der Webseiten (XML-Parsing ect.) genutzt werden. Bitte achten Sie darauf, dass alle verwendete Abhängigkeiten in der `pom.xml` enthalten sind.

2.6 Annahmen

Bei der Entwicklung des Algorithmus dürfen Sie von folgenden Annahmen (Vereinfachungen) ausgehen:

- Sie dürfen sich bei der Durchsuchung der Webseiten auf Standard-HTML-Seiten mit den `<img ...>`-Tags beschränken. Eventuell könnte Ihr Browser weitere Abbildungen dynamisch via JavaScript nachladen. Eine derartige Dynamik und Komplexität müssen Sie nicht berücksichtigen.

²https://www.w3schools.com/html/html_images.asp

- Bitte laden Sie stets ausschließlich die Abbildungen auf der Website der übergebenen URLs herunter. Folgen Sie keinen Links auf den HTML-Seiten.
- Der Fokus der Programmieraufgabe liegt in der Implementierung der Parallelisierung. Sollten Sie beim Download der Abbildungen auf unvorhergesehenen Hürden wie falsche Dateieindungen, kryptische Dateinamen oder ungültige Abbildungslinks stoßen, dürfen Sie diese ignorieren.

3 Aufgabenteile

Die Prüfungsleistung besteht aus drei Aufgabenteilen, die im folgenden beschrieben sind:

3.1 Entwurf und Planung des Programms

Vermitteln Sie in einem Dokument die Architektur und Struktur Ihres Programms. Insbesondere soll darauf eingegangen werden, wie das Programm modularisiert ist und über welche Kanäle diese Module die Daten untereinander austauschen. Beschreiben Sie die Bereiche die Sie parallelisieren wollen und welche Algorithmic Strategy Patterns und Implementation Strategy Patterns Sie einsetzen.

Die Beschreibung in dem Dokument soll auf hohem Niveau erfolgen und soll die Idee hinter Ihrer Implementierung anschaulich darstellen. Implementierungsdetails (alle Attribute, Datentypen, ...) können weggelassen werden. Gerne dürfen Sie Grafiken zur Visualisierung verwenden. Erwartet werden ca. 2 Seiten (Textinhalt) zzgl. Abbildungen.

3.2 Programmcode

Der gesamte Java-Programmcode (nicht die kompilierten Binaries) ist als ZIP-Archiv bereitzustellen. Der Programmcode soll unter anderem enthalten:

- Java-Doc für alle Methoden
- Unittests (JUnit 5)

3.3 Beantwortung von Fragen

Beantworten Sie in einem Dokument folgende Fragen:

1. Welche Teile des Programms müssen gegenüber parallelem Zugriff geschützt werden?
2. Wie stellen Sie den Schutz gegenüber dem parallelen Zugriff an den genannten Stellen sicher? Welche Alternativen hätte es gegeben? Warum haben Sie sich für diese Variante des Schutzes entschieden?
3. Nennen Sie die Objekte und Datenstrukturen, die pro Thread zusätzlich im Arbeitsspeicher (Heap oder Stack) vorgehalten werden müssen. Auf JVM-interne Details zum erforderlichen Speicher für einen Thread müssen Sie nicht eingehen.

4 Upload

Laden Sie im Moodle exakt **ein** ZIP-Archiv hoch. Verwenden Sie Ihren Namen als Dateinamen für das ZIP-Archiv.

Das ZIP-Archiv soll genau die folgenden drei Dateien/Ordner enthalten:

- **Entwurf.pdf**: PDF-Dokument mit dem Entwurf und der Planung des Programms aus 3.1.
- **Code**: Ordner mit gesamtem Programmcode inkl. `pom.xml`, Java-Doc und Tests aus 3.2. Die ZIP-Datei soll nicht die kompilierten Java-Klassen (*.class-Dateien) enthalten.
- **Fragen.pdf**: PDF-Dokument mit der Beantwortung der Fragen aus 3.3. Achten Sie darauf, dass die Zuordnung von Antwort zur jeweiligen Frage klar ist (z.B. durch Nummerierung oder Kopieren der Frage).

5 Bewertungskriterien für Programmcode

Für die Bewertung des Programmcodes werden insbesondere folgende Kriterien herangezogen:

- Funktionalität
- Korrektheit
- Thread-Sicherheit (Thread-Safety)
- Geschriebene Tests
- Abdeckung von Grenzfällen durch die Tests
- Dokumentation des Programmcodes (Java-Doc)
- Einheitlicher/sinnvoller Code-Style
- Aussagekräftige und nachvollziehbare Namensschema (Klassen, Methoden, Variablen)
- Strukturierung des Programms (sinnvolle Aufteilung der Klassen und Methoden)